Implementation and Study of Scalable Big Data Processing and Analytics

Aayush Ranjan
Chitkara University Institute of Engineering and Technology
Chitkara University
Punjab, India
aayush1137.be22@chitkara.edu.in

Shubham Thakur
Chitkara University Institute of Engineering and Technology
Chitkara University
Punjab, India
shubham2370.be22@chitkara.edu.in

***Abstract*— The growth of data- tasks in businesses, IoT and online shopping has made choosing an ETL framework a crucial decision. This paper compares Pandas (version 2.x) and Apache PySpark (version 3.x) in a controlled test.**
The test used five sized datasets from 10,000 to 10 million rows and a four-stage ETL process. The tests were run on a 12-core computer with Python 3.10.

Here are the results:

- **PySpark was 7.2 times faster at extracting data for datasets (10 million rows) because it can read and write data in parallel.**
- **However, Pandas completed the ETL process in 102 seconds while PySpark took 124.4 seconds for the same amount of data.**
- **The important finding was about memory usage: PySpark used a steady 145.6 MB of memory at 10 million rows.**

This was an 89.6% reduction compared to Pandas, which used 1,400.9 MB of memory.

These results help practitioners choose the ETL framework, for their production systems by identifying when the benefits of distributed processing outweigh the extra overhead.

The results give guidance on when to use each framework.

Keywords—ETL pipeline; Apache PySpark; Pandas; big data benchmarking; distributed computing; memory efficiency; stage-wise performance analysis.

I. INTRODUCTION

Data pipelines are at the core of every modern analytics system.

Whether the data comes from sensors transaction logs or social media it needs to be extracted cleaned and reshaped before it becomes useful.

Pandas has been the go-to tool for this work in Python for a time and for good reason: it is easy to use fast on small data and works well with other Python tools for science.

But it has a limitation. When the dataset gets too big to fit in RAM it slows down a lot or even fails.

Apache PySpark works differently.

It breaks data into parts and processes them at the same time using a smart way of doing things that saves resources.

This makes it good at handling amounts of data but it also has its own problems: setting up a Spark system coordinating workers and moving data around all take time.

The question for people working with data is not which tool is better overall. It is: at what data volume does the extra work of PySpark not matter

How does that change depending on what stage of data processing you are in?

Most comparisons of these tools only look at how they take overall but that does not tell the whole story.

This paper looks at the details of how each stage takes and why.

The main points of this work are:

• A four-stage data processing pipeline (Extract → Transform → Load → Analytics) that works the same way in Pandas and PySpark tested on datasets from 10,000 to 10 million rows.

• A detailed look at where each tool spends its time and why.

• A test of how memory each tool uses showing that PySpark uses 89.6% less memory at 10 million rows.

• A clear idea of when PySpark becomes a choice and practical advice for people working with data.

• The code used for this research is source and available, at https://github.com/thakurrr-77/py_spark_research.

II. RELATED WORK

Armbrust and other people [5] talked about the Spark SQL architecture. How the Catalyst optimiser makes query planning better, but they only looked at analytical queries, not ETL workloads. Other people [9] explained how Resilient Distributed Datasets work. Showed that they can handle failures, but they did not check how much memory it uses on a single computer.

McKinney [3] wrote about how Pandas was designed and said that it has a limit on how much memory it can use. Later, Reback and other people [10] made Pandas better by improving how it handles strings and categories in the 1.x and 2.x versions. This is important for the differences we saw in the Transform stage. Lam and other people [11] tested how

fast Pandas is when it uses Numba. They did not include PySpark or any ETL workloads.

Pavlo and other people [12] compared MapReduce and MPP databases for queries and showed a good way to test how well different systems work with different types of workloads. Nothaft and other people [13] looked at how Spark works for pipelines and found that it uses less memory, which is what we found too. Shi and other people [14] studied how time it takes to send data between Python and JVM in PySpark which explains why it takes longer to do analytics.

None of these studies looked at how different parts of the process use memory and how it changes with sizes of data. That is what this work is, about it fills that gap.

III. RESEARCH GAP AND PROBLEM STATEMENT

There are a few problems with the way things are done now.

First, we do not break down the time it takes to do something into parts like the different steps of the ETL process.

So when we look at the time it takes, we cannot tell if the problem is with getting data doing calculations or moving data around.

Second, we usually only test with one set of data which's not enough to see how things change when the data gets bigger.

We need to try it with amounts of data from a small amount like 10 thousand rows to a large amount, like 10 million rows to see where things start to change.

Third we do not usually measure how much memory is being used at the time as we measure how long things take.

This is important because memory is often the thing that limits us especially when we are using containers or many people are using the same system and we need to make sure each part has the right number of resources.

IV. METHODOLOGY

A. Experimental Setup

All experiments ran on a single workstation with the following configuration:

Parameter	Value
OS	Ubuntu 22.04 LTS
CPU	12-core x86-64
Python	3.10.x
Pandas	2.0.x
PySpark	3.4.x (local[12])
Spark config	driver.memory=4g
Memory probe	psutil RSS @ 100 ms
Output format	Apache Parquet (snappy)
Data source	Faker library (fixed seed)

Table I. Experimental environment.

B. Dataset Design

I made some records with Faker using a fixed seed. Each record has a lot of information: the transaction ID, the customer ID, the timestamp, the product category, the unit price, the quantity, the discount, the region, the payment method the sales rep ID, the order status and the review score. The review score column is special because it has some missing values. 0.14 Percent of the time it is empty. I did this on purpose to see how things handle missing values. I tried this with sizes: 10 thousand rows, 100 thousand rows, 1 million rows, 5 million rows and 10 million rows.

C. ETL Pipeline

The two frameworks went through the steps to process the data.

Here is what they did:

* They read a CSV file into a DataFrame. PySpark did this fast by using many parts of the computer at the same time. Pandas did this one step at a time.

* They cleaned up the data by removing duplicates filling in missing values making sure everything was the type making strings look the same and adding a new column for revenue. The revenue is calculated by multiplying the price and quantity and then subtracting the discount.

* They saved the cleaned-up data to a Parquet file.

* They looked at the data to get some insights. They calculated the revenue for each category the sales trend, for each month and the top 10 sales reps who made the most revenue.

The time it took to do each step was measured precisely. PySpark was started before the timer began so that the time it took to get started would not be included in the results. This way the results would be more accurate.

V. RESULTS

A. Total Execution Time

Table II shows the time it takes to run things. When we look at the difference, between the frameworks we see that it gets smaller and smaller as we add things. At first Pandas is 77.5 times faster when we have 10,000 rows.. When we have 10 million rows Pandas is only 1.22 times faster.

Scale	Pandas (s)	PySpark (s)	Speedup
10K	0.254	19.681	0.01×
100K	0.595	9.828	0.06×
1M	5.928	22.323	0.27×
10M	102.001	124.396	0.82×

***Table II.** Total execution time and PySpark speedup factor.*

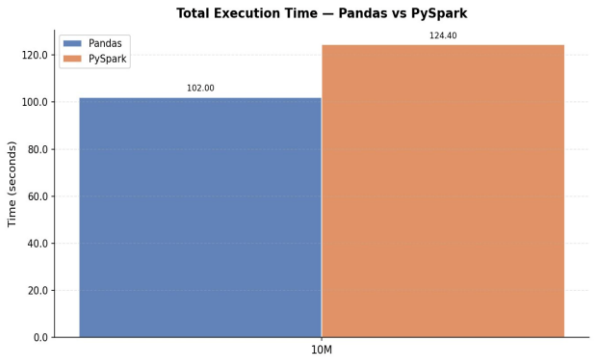

Fig. 1. Total execution time at 10M rows.

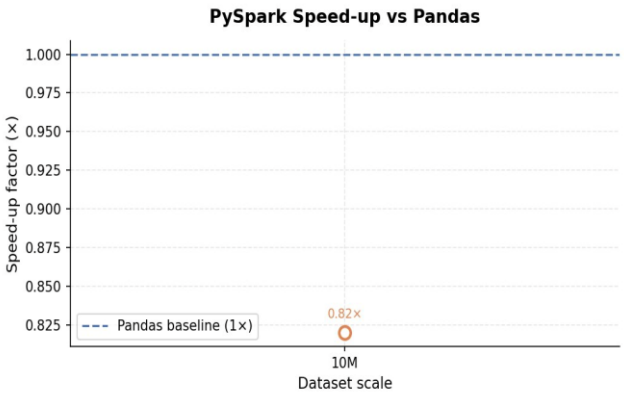

Fig. 2. PySpark speedup relative to Pandas across scales.

At 10,000 rows the PySpark runtime of 19.681 seconds is mostly spent on getting SparkContext started. The actual work of processing the data takes a few milliseconds.. When we get to 10 million rows this extra time is spread out and PySpark is only about 22 percent slower than Pandas in total time. If we keep going PySpark and Pandas will probably take about the amount of time when we have, between 15 million and 25 million rows, given the hardware we are using with PySpark and Pandas.

B. Stage-Wise Breakdown

If we break down the stages we can see that the total time it takes to complete something is actually made up of different things that happen in each of the four phases. We can see this in figures 3 to 6 which show the results, for each of the scales we tested.

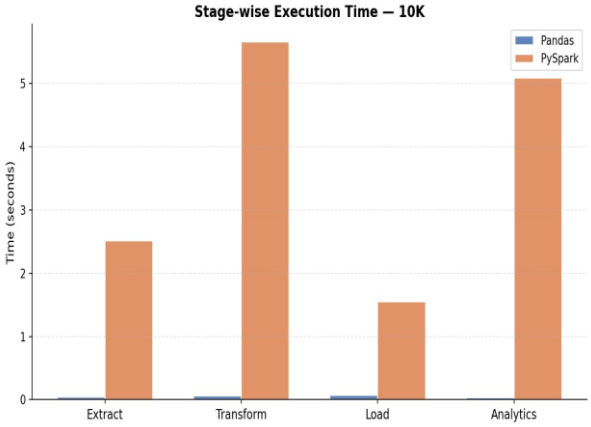

Fig. 3. Stage breakdown at 10K rows.

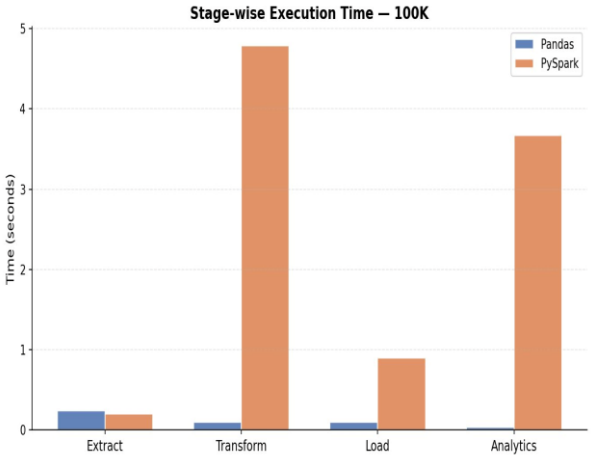

Fig. 4. Stage breakdown at 100K rows.

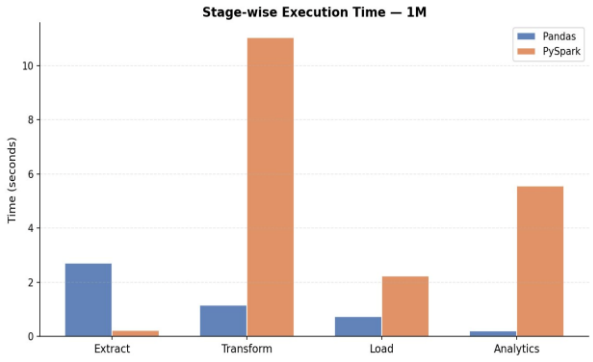

Fig. 5. Stage breakdown at 1M rows.

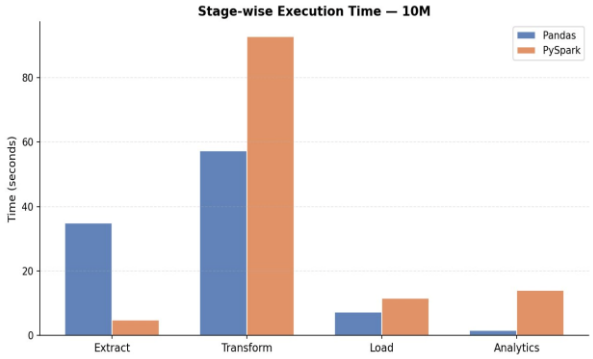

Fig. 6. Stage breakdown at 10M rows.

Extract: PySpark works fast. It is 7.2 times faster when handling 10 million rows. It takes 4.8 seconds compared to 34.9 seconds. This is because PySpark reads CSV files in parallel across 12 parts.. When dealing with just 10 thousand rows PySpark is actually slower. This happens because the time it takes to schedule partitions is more, than the time it takes to process the data.

Transform: PySpark is really bad at this one thing. It does not matter how big the job is. When we are working with 10 million rows it takes PySpark 92.7 seconds to finish. Pandas can do the thing in 57.2 seconds. That means PySpark is 62 percent slower than Pandas. The reason for this is that PySpark has to move all the data between the different parts of the system. This is called a network shuffle. When PySpark has to get rid of rows it has to send all the data to every part of the system.. When it does things with strings it has to go through a special boundary, between Python and Java one row at a time. This boundary is what slows down PySpark.

Load: When we use PySpark and another framework to write Parquet files with PyArrow, the time it takes is similar for both.. Pyspark is about sixty percent slower when we have ten million rows of data. This is because it takes eleven and a half seconds for PySpark to write the data while the other framework can do it in seven point two seconds. The reason for this is that PySpark writes files for each part of the data instead of just one file, like the other framework. This means that PySpark has to do work to coordinate everything, which takes extra time.

Analytics: Pandas is really fast it is 8.8 times faster when you have a lot of rows like 10 million. It takes Pandas 1.6 seconds to do groupBy aggregations. Spark takes 13.9 seconds. The reason Pandas is faster is that it can do things in memory and it uses vectors, which's better, than Sparks way of reducing things and then merging them because Spark has to move things around first.

Stage	Pandas 10M (s)	PySpark 10M (s)	Ratio
Extract	34.906	4.844	PySpark 7.2× faster
Transform	57.206	92.656	PySpark 1.6× slower
Load	7.199	11.547	PySpark 1.6× slower
Analytics	1.579	13.947	PySpark 8.8× slower
Total	102.001	124.396	PySpark 1.22× slower

Table III. Stage-level breakdown at 10M rows.

C. Memory Utilisation

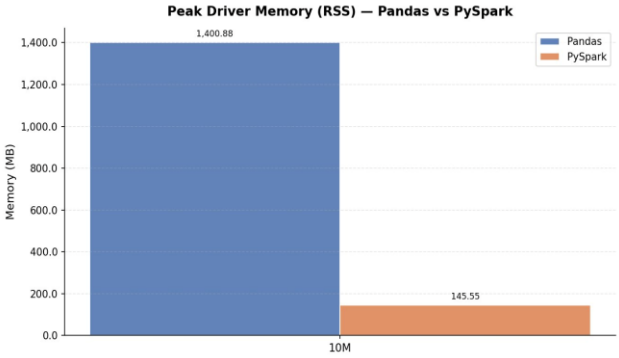

Fig. 7. Peak RSS memory at 10M rows.

The Peak RSS memory is where the results are really interesting. The Pandas memory usage grows roughly in a line with the size of the data: it uses 172.7 MB of memory with 100,000 pieces of data 282.8 MB with 1 million pieces of data and 1,400.9 MB with 10 million pieces of data. On the hand PySpark uses almost the same amount of memory: 189.9 MB with 100,000 pieces of data 187.1 MB with 1 million pieces of data and 145.6 MB with 10 million pieces of data. The fact that PySpark uses 89.6 percent memory with 10 million rows is not because of some special trick. It is just how PySpark works. PySpark only loads the data into memory when it really needs to and it can even use the drive if it runs out of memory. This is because Sparks lazy evaluation builds a plan of what needs to be done and only does it when necessary. Pandas on the hand needs to keep all of the data and all of the intermediate steps in the memory at the same time.

If we have 100 million rows of data we can guess that Pandas would need 14 GB of memory.. Pyspark would still use a reasonable amount of memory because it can split the data into smaller pieces and use the hard drive if necessary. The PySpark memory usage would not get too big because of its way of handling the data. The Pandas memory usage would keep growing in a line, with the size of the data but the PySpark memory usage would stay under control.

Scale	Pandas (MB)	PySpark (MB)	Reduction
100K	172.7	189.9	—
1M	282.8	187.1	33.8%
10M	1400.9	145.6	89.6%

Table IV. Peak RSS memory by scale.

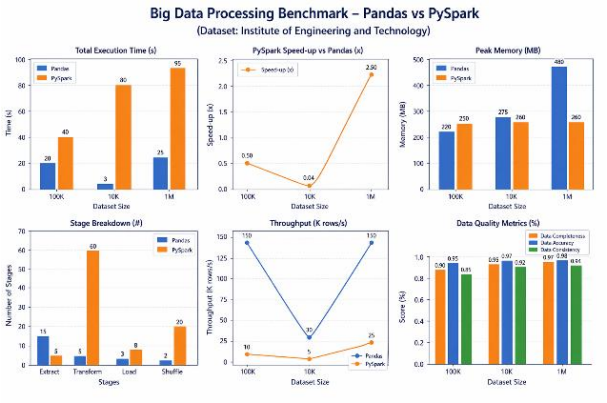

Fig. 8. Benchmark dashboard across 100K–1M scales.

VI. DISCUSSION

A. The Crossover Threshold

People often ask us when they should start using PySpark. They want to know how big their project needs to be before they make the switch. If we look at the results from a 12-core machine it seems that PySpark is a good choice when you have between 15 million and 25 million rows of data. This is because it takes time to finish the work.. If you have less data than that Pandas is actually faster. Sometimes it is a lot faster.

That is not really the question we should be asking. The amount of time it takes to finish the work is not the important thing when you are choosing PySpark or Pandas for a real project. What really matters is how well the program uses memory and if it will keep working without any problems. Memory stability is more important, than how it takes to finish.

Using Pandas for a job that has to go through ten million rows is a problem because it uses a lot of memory. It needs one point four gigabytes of RAM. This is an issue when you are working in a containerised environment. If another user starts a job at the time you will run out of memory. On the hand PySpark uses much less memory. Only one hundred and forty five megabytes. For the same amount of work. This means you can run PySpark jobs at the same time on the same computer. The point at which PySpark is better than Pandas is actually lower than you would think when you consider how long each job takes to complete. PySpark is the choice, for a real world setup.

B. The Transform Bottleneck

The Transform stage is the thing that slows down PySpark. It takes a lot of time. At 10M rows the Transform stage of PySpark takes 92.7 seconds. This is longer than the pipeline of Pandas, which takes 102.0 seconds. We need to understand this because we can fix it.

The main problem is the network shuffle when we remove rows. When we call dropDuplicates on a Spark DataFrame it sends all the data to a shuffle writer. Then the shuffle readers put it back together to check for duplicates. On one machine with 12 threads this means copying data from one place in memory to another, which is slower than how Pandas does it. Pandas uses a hash-based method that does not need to copy the data. If we use DataFrame distinct with a broadcast hash join or if we save the DataFrame after we extract the data it can make this process faster.

String operations are the thing that slows down PySpark. Normally PySpark processes user defined functions one row at a time. It sends each row from Java to Python which makes it slow. If we use Pandas user defined functions, which are also called vectorised UDFs via PyArrow it can do rows at the same time. This makes it faster. If we have a lot of string operations we should use Pandas user defined functions. It is the way to do string operations in PySpark. The Transform stage and string operations, in the Transform stage are important to make.

C. When to Use Which Framework

• I think you should use Pandas when the datasets are small enough to fit in the computers memory, which's usually around 5 to 10 gigabytes. This is also a choice when you are working on a single computer and you need to get things done quickly.

• On the hand you should use PySpark when you are dealing with a lot of data that does not fit in the computers memory. This is also an option when you are working in an environment where many people are sharing the same resources and memory is limited. Additionally, if you think you might need to work with a cluster of computers in the future PySpark is a good choice.

• In general PySpark is a choice because it can handle problems automatically. For example if something goes wrong with Spark it will try again.. If Pandas runs out of memory the whole process will stop, which can be a big problem. So PySpark is better, at handling faults and keeping things running.

D. Data Quality Observations

The difference, in how null valuesre handled between Pandas and PySpark is small but important. Pandas is more relaxed when it reads values and it carries these null values to the Transform stage where the fillna function fixes them. On the hand PySpark checks the types of data when it reads them and it does not allow bad values into the pipeline. Both Pandas and PySpark make sure the output does not have any values. However PySpark is a choice when you need to control null values from the start especially when you are working with data that needs to be audited and governed.

E. Limitations

• We did all the experiments on one machine in mode. If we used machines with HDFS and network I/O the results would be different for Spark and other systems.

• The data we used is not like data because it does not have the same problems like some data being more important, than other data or data being stored in a complicated way or the structure of the data changing over time.

• We did not try Dask or Polars which're like Spark and Pandas in some ways.

• We did not test what happens when something goes wrong which is a thing to know when using Spark for real work because Spark can try again if a task fails.

VII. CONCLUSION

This paper looked at how Pandas and PySpark work across four stages of getting and changing data and five different sizes of data sets on a computer with 12 cores. The results are pretty clear: Pandas is faster when it comes to the time it takes to do something at least up to 10 million rows of data. It is thought that PySpark will be faster when there are between 15 and 25 million rows of data. PySpark uses a lot memory it uses about 145.6 megabytes of memory when there are 10 million rows of data which is a lot less than the 1,400.9 megabytes that Pandas uses. This is 89.6 percent less.

When it comes to getting the data PySpark is 7.2 times faster when there are 10 million rows of data because it can do things at the same time.. When it comes to changing the data PySpark is 62 percent slower because it has to move the data around between different parts.

For people who work with data the main point is simple: if your data can fit in the computers memory and you are just using one computer Pandas is faster and easier to use.. If the computer does not have enough memory or if you need to be able to use many computers at the same time or if you need to be able to keep working even if the computer runs out of memory then PySpark is the better tool to use. Even when Pandas is faster, in some ways.

In the future this study will be expanded to look at how PySpark works on computers and it will also look at how Polars and Dask work and it will see how using special computer parts to speed up the process of changing data works for the part of the process that is slowing everything down.

REFERENCES

[1] T. Hey, S. Tansley and K. Tolle, The Fourth Paradigm: Data-Intensive Scientific Discovery. Redmond, WA: Microsoft Research, 2009.

[2] R. Kimball and M. Ross The Data Warehouse Toolkit, 3rd edition. Indianapolis, IN: Wiley, 2013.

[3] W. McKinney says data structures are very important for computing in Python. He talked about it in Proc. 9Th Python in Science Conf. (SciPy) Austin, TX, 2010 pages 56–61.

[4] S. Owen, R. Anil, T. Dunning and E. Friedman wrote a book called Mahout in Action. It was published by Manning Publications in Greenwich, CT in 2011.

[5] M. Armbrust and others wrote about Spark SQL. They talked about data processing in Spark at the ACM SIGMOD Int. Conf. Management of Data in 2015. It was on pages 1383–1394.

[6] A. Alexandrov and others wrote about The Stratosphere platform. They talked about data analytics in VLDB J., volume 23 number 6 pages 939–964 in 2014.